

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 18 tháng 6 năm 2026

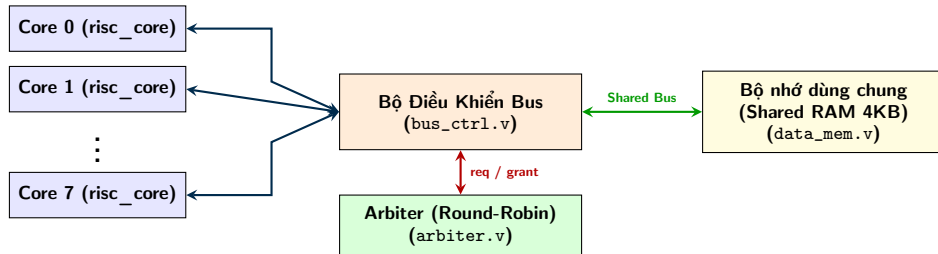
1 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

1 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

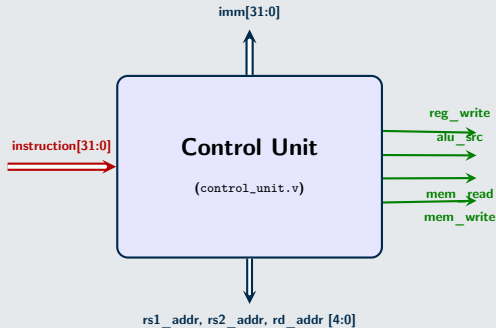
1 Thiết Kế Hệ Thống (Design)

- Cấu trúc tổng thể
- Kiến trúc Pipeline và Logic Điều khiển

Control Unit

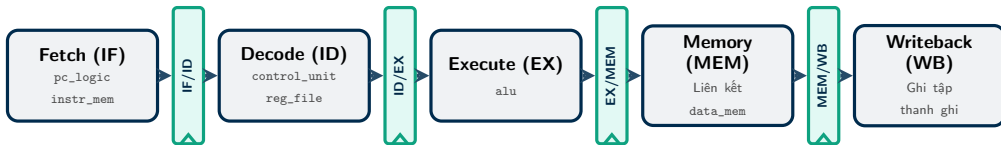
- **Nhiệm vụ:** Sử dụng Combinational Logic để giải mã các trường Opcode, Funct3, Funct7 của Instruction và lập tức sinh các tín hiệu điều khiển cho hệ thống Datapath.

Sơ đồ Input Output Control Unit



Phân bố phần cứng trên 5 tầng Pipeline

Hệ thống **Pipeline Registers** đóng vai trò là các vách ngăn đồng bộ, cô lập dữ liệu giữa các giai đoạn xử lý:



Nguyên lý điều khiển luồng trên các Thanh ghi đệm Pipeline

- **Clock Synchronization:** Dữ liệu/lệnh từ tầng trước chỉ được phép dịch chuyển sang tầng sau chính xác tại sườn lên của xung Clock (posedge clk).
- **Pipeline register:** Giữ cố định trạng thái xử lý độc lập của 5 lệnh khác nhau cùng lúc, ngăn chặn hiện tượng lẫn át hay trộn lẫn dữ liệu giữa các tầng Datapath.
- **Pipeline Interlock:** Là nơi trực tiếp tiếp nhận các lệnh (hold để tạo Stall, hoặc clear để tạo Flush/Bubble) từ bộ điều khiển trung tâm.

PIPELINE REGISTERS: Fetch / Decode (IF/ID Register)

- **Vai trò:** Chốt giữ Instruction 32-bit vừa đọc ra từ Instruction Memory và địa chỉ PC tương ứng để chuyển giao sang giai đoạn Decode.
- **Cơ chế Hazard:** Nhận tín hiệu hold khi kích hoạt Load-Use Stall hoặc Memory Stall để đóng băng trạng thái; nhận tín hiệu clear để thực hiện Flush lệnh rác khi rẽ nhánh Taken sai.

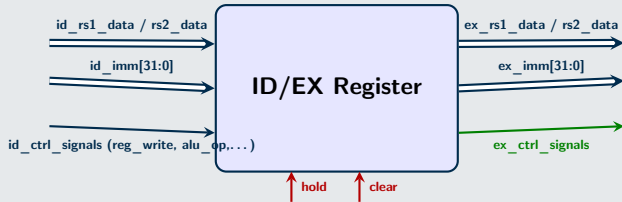
Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_if_id.v



PIPELINE REGISTERS Decode / Execute (ID/EX Register)

- **Vai trò:** Tiếp nhận và chuyển tiếp toàn bộ các tín hiệu điều khiển từ Control Unit, giá trị các toán hạng đọc từ Register File (`rs1_data`, `rs2_data`) và số Immediate đã mở rộng sang giai đoạn Execute cho ALU.
- **Cơ chế Hazard:** Nhận tín hiệu `clear = 1` để chủ động chèn một Bubble (NOP) vào tầng Execute khi xảy ra xung đột Load-Use.

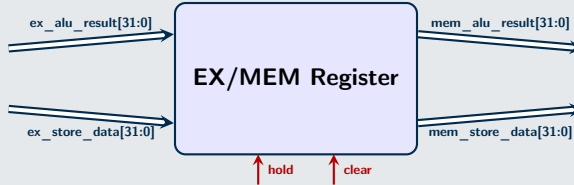
Sơ đồ cổng giao tiếp (Black Box) - `pipe_reg_id_ex.v`



PIPELINE REGISTERS Execute / Memory (EX/MEM Register)

- **Vai trò:** Chốt giữ kết quả tính toán từ đầu ra của ALU, dữ liệu cần ghi vào Data Memory (rs2_data) của lệnh Store, và địa chỉ thanh ghi đích rd.
- **Cơ chế Hazard:** Nhận tín hiệu hold từ mạch Memory Stall (mem_wait) nhằm giữ nguyên thông tin khi Shared Bus đang bận phân xử quyền truy cập bộ nhớ đa nhân.

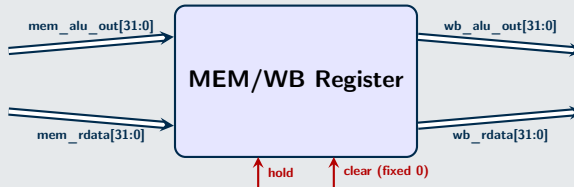
Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_ex_mem.v



PIPELINE REGISTERS Memory / Writeback (MEM/WB Register)

- **Vai trò:** Chốt giữ dữ liệu đọc lên từ Data Memory (rdata) hoặc kết quả trực tiếp từ ALU để sẵn sàng đưa về cổng ghi của Register File.
- **Cơ chế Hazard (Đã vá lỗi):** Loại bỏ hoàn toàn chân clear độc hại, chuyển sang sử dụng chân hold đồng bộ. Khi gặp Memory Stall, dữ liệu tầng này đứng im để bảo toàn dữ liệu, tránh lỗi mất thông tin trên Pipeline.

Sơ đồ cổng giao tiếp (Black Box) - pipe_reg_mem_wb.v



Hazard Detection & Forwarding

- **Tư duy tích hợp:** Gom toàn bộ logic phát hiện Load-Use Stall, cứu dữ liệu (Forwarding) và quản lý bộ nhớ bận (mem_wait) vào một bộ điều khiển tập trung để tối ưu tài nguyên.

Sơ đồ khối điều khiển trung tâm hệ thống - Hazard Controller



Nguyên tắc đóng gói:

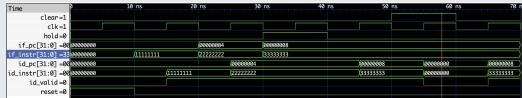
- Giữ nguyên 100% giao tiếp Ports ra ngoài hệ thống Bus và mô hình đa nhân (8 cores) của nhóm.
- Bên trong ruột gọi (instantiate) 6 module con độc lập và thực hiện kết nối bằng dây dẫn (wire).

Cấu trúc đầu nối Hazard Controller trong Core

```
1 // --- PASTE DOAN CODE INSTANTIATE HAZARD_CONTROLLER  
  TRONG CORE TẠI ĐÂY ---
```

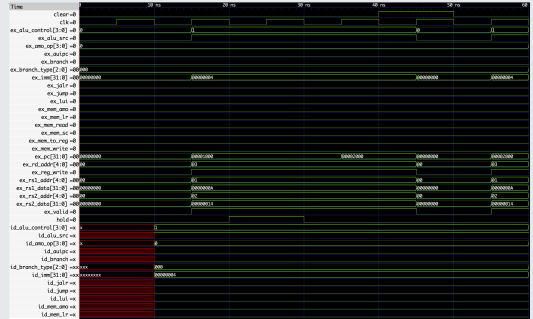
Pipeline Register Unit Testing - IF/ID và ID/EX

1. Kết quả mô phỏng pipe_reg_if_id



Phân tích: Minh chứng phản hồi chính xác khi tín hiệu hold dựng lên, giá trị PC và lệnh được khóa cứng qua xung clock.

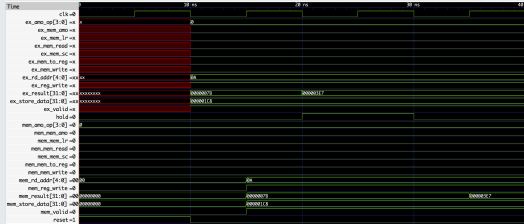
2. Kết quả mô phỏng pipe_reg_id_ex



Phân tích: Tín hiệu clear (Flush) kích hoạt thành công, xóa sạch các cờ điều khiển rác về bằng 0 ngay chu kỳ sau.

Pipeline Register Unit Testing EX/MEM và MEM/WB

3. Kết quả mô phỏng pipe_reg_ex_mem



Phân tích: Dữ liệu địa chỉ và toán hạng lưu trữ từ ALU được đồng bộ đẩy sang bộ nhớ chính xác theo sườn lên của Clock.

4. Kết quả mô phỏng pipe_reg_mem_wb



Phân tích: Sửa lỗi sập hầm bằng chân hold thay thế clear. Khi gặp mem_wait, tăng ghi được giữ nguyên, bảo toàn dữ liệu.